DATA STRUCTURES

ETCCDS202

Assignment-1

Topic: Foundations & Algorithmic Analysis (Unit-1)

Short Analysis Report

Submitted By: Meghna Kumar
Roll No.: 2501730214
Program: B.Tech CSE (AI & ML)
Section: A
Faculty Name: Ms Neetu Chauhan
Submitted on: 04-03-2026

Assignment Overview:

This assignment focuses on understanding Data Structures, Abstract Data Types (ADTs), recursion, algorithmic analysis, and basic algorithm paradigms. A Python toolkit named Algorithmic Efficiency & Recursion Toolkit (AERT) has been implemented to demonstrate these concepts along with a written analysis of their efficiency.

PART A: STACK ADT

1. **What is an ADT?**

   An Abstract Data Type (ADT) defines a data type by specifying the operations that can be performed on it without specifying how these operations are implemented. Stack ADT supports the following operations: - push(x), pop(), peek(), is_empty(), size()

2. **Why stack operations are usually O(1) (constant time)?**

   Stack operations such as push, pop and peek take constant time O(1) because they operate only on the top element of the stack without any iteration.

3. **Use your StackADT in at least one place:**

   The StackADT is used in this assignment to store and record the sequence of moves in the Tower of Hanoi problem.

PART B: FACTORIAL AND FIBONACCI (RECURSION + COMPLEXITY)

1. **Factorial (Recursive)**
   The factorial function is implemented using recursion with the base cases 0! = 1 and 1! = 1.
   - Time Complexity: O(n)
   - Space Complexity: O(n)
   - Big-Ω: O(n)
   - Big-Θ: O(n)
2. **Fibonacci (Naive Recursive)**

The naive Fibonacci implementation directly follows the recursive definition of Fibonacci numbers.

- Time Complexity: $O(2^n)$
- Space Complexity: $O(n)$

3. **Fibonacci (Memoized Recursive)**
   The memoized Fibonacci implementation stores previously computed values to avoid redundant calculations.
   - Time Complexity: $O(n)$
   - Space Complexity: $O(n)$

   The call counter clearly shows a significant reduction in recursive calls when memorization is used.

4. **Why naive Fibonacci becomes slow for large n?**
   The Naive Fibonacci becomes slow for large values of n because the same subproblems are solved repeatedly.

PART C: TOWER OF HANOI (RECURSION + TRACE)

1. **Why number of moves become $2^n - 1$?**
   For n disks, the total number of moves required is 2^n-1.
2. **Complexity Analysis:**
   - Time Complexity: $O(2^n)$
   - Space Complexity: $O(n)$

   The recursion depth corresponds to the number of disks, resulting in linear space usage due to the call stack.

PART D: RECURSIVE BINARY SEARCH (DIVIDE & CONQUER + RECURRENCE)

1. Binary Search is implemented recursively to search for a key in a sorted way.
2. **Recurrence Relation:**
   $T(n)=T(n/2)+O(1)$
3. **Complexity Analysis:**
   - Time Complexity: $O(\log n)$

- Best Case: The result is found immediately (e.g., middle element in binary search)
  Time Complexity: O(1)
- Average Case: The result is found after a few steps.
  Time Complexity: O(log n)
- Worst Case: The maximum number of steps are required (element at last level or not found).
  Time Complexity: O(log n)

The function correctly handles cases where the array is empty or the key is not present.

PART E: SHORT PARADIGM REFLECTION

1. **Binary Search is Divide & Conquer**
   Binary Search uses Divide & Conquer approach because it:
   - Divides the sorted array into two halves.
   - Conquers by searching only the relevant half.
   - Combines results implicitly by returning the found index.

   The problem size is reduced by half at each step.

2. **Fibonacci memoized is Dynamic Programming**
   Fibonacci with memoization is Dynamic Programming because:
   - It has overlapping subproblems (same Fibonacci values are computed repeatedly).
   - Memoization stores already computed results, avoiding repeated calculations. This reduces time complexity from exponential to linear.
3. **Tower of Hanoi uses recursion**
   Tower of Hanoi uses recursion because:
   - Base Case: If there is only one disk, move it directly to the destination rod.
   - Recursive Idea: Move n-1 disks to helper rod, move the largest disk, then move the n-1 disks onto it.

The problem is solved by breaking it into a smaller subproblems of the same type.

4. **Real-life examples:**
    - Divide & Conquer: Searching for a word in a dictionary by opening it in the middle and narrowing it down.
    - Dynamic Programming: Planning a trip by remembering past routes to avoid repeating mistakes.
    - Greedy: Giving change by always using the largest available coin first.